Reviewer Pack — Minimal Rank of Tropical Modules and Circuit Monotone Width

Entry f3f8847f2523 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 09:35:59 UTC

Minimal Rank of Tropical Modules and Circuit Monotone Width

Entry ID: `f3f8847f2523` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-05 09:35:59 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Boolean Circuit Complexity

Statement:

> For every Boolean circuit C with monotone width w(C), the minimal rank of its associated tropical module, denoted as r_trop(C), is O(w(C)). Moreover, for a given instance of size n ≤ 40, if C is satisfiable, then r_trop(C) = Θ(w(C)).

Rationale (proposer's reasoning):

> Tropical modules provide a geometric interpretation of circuits, and their ranks could potentially reveal deep connections between the structure of a circuit and its complexity. This conjecture aims to bridge the gap between tropical geometry and Boolean circuit complexity by suggesting that the rank of a tropical module is proportional to the monotone width of a circuit, which is an important measure in circuit complexity.

Taxonomy category: `TROPICAL_FOURIER_ANALYSIS` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `2d90e1b9524c9937`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all Boolean circuits C with monotone width w(C) ≤ 40, the ratio of the mean rank of the associated tropical module r_trop(C) over the monotone width w(C) is less than or equal to 1.5 AND the correlation coefficient between r_trop(C) and w(C) across 30 random seeds is greater than 0.8. The conjecture is falsified if either condition fails.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `tropical geometry AND Boolean circuit monotone width` - `minimal rank tropical modules circuit monotone width` - `Boolean circuit satisfiability minimal rank tropical module`

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_circuit(n):
        if n == 1:
            return [random.choice([0, 1])]
        else:
            left = generate_boolean_circuit(n // 2)
            right = generate_boolean_circuit(n - n // 2)
            return [(x and y) for x in left] + [(x or y) for y in right]

    def monotone_width(circuit):
        if len(circuit) == 1:
            return 1
        else:
            return max(monotone_width(circuit[:len(circuit)//2]), monotone_width(circuit[len(circu

    def tropical_module(circuit):
        n = len(circuit)
        M = [[0] * (n + 1) for _ in range(n + 1)]
        for i in range(1, n + 1):
            M[i][i-1] = circuit[i-1]
        for j in range(2, n + 1):
            for i in range(j - 1, 0, -1):
                M[i][j-1] = max(M[i][j-2], M[i+1][j-2])
        return M
```

```
def rank(matrix):
    m, n = len(matrix), len(matrix[0])
    if m == 0 or n == 0:
        return 0
    for i in range(m):
        for j in range(n):
            if matrix[i][j] != 0:
                pivot_row = i
                break
        else:
            continue
        for k in range(j, n):
            matrix[pivot_row][k], matrix[i][k] = matrix[i][k], matrix[pivot_row][k]
        for l in range(m):
            if l != pivot_row and matrix[l][j] != 0:
                factor = -matrix[l][j] / matrix[pivot_row][j]
                for k in range(j, n):
                    matrix[l][k] += factor * matrix[pivot_row][k]
    return sum(1 for row in matrix if any(row[i] != 0 for i in range(n)))

total_ranks = []
total_widths = []
instances_tested = 0
n_max = 0

for n in [5, 10, 15, 20, 30, 40]:
    for _ in range(5):  # Ensure at least 30 instances per seed
        circuit = generate_boolean_circuit(n)
        width = monotone_width(circuit)
        if width == 0:
            continue
        M = tropical_module(circuit)
        r_trop = rank(M)
        total_ranks.append(r_trop)
        total_widths.append(width)
        instances_tested += 1
        n_max = max(n_max, n)

mean_rank = sum(total_ranks) / len(total_ranks)
mean_width = sum(total_widths) / len(total_widths)
correlation_coefficient = (sum((r - mean_rank) * (w - mean_width) for r, w in zip(total_ranks,
                           math.sqrt(sum((r - mean_rank)**2 for r in total_ranks)) *
                           math.sqrt(sum((w - mean_width)**2 for w in total_widths)))

conjecture_holds = correlation_coefficient > 0.8 and (mean_rank / mean_width) <= 1.5
counterexample = "" if conjecture_holds else "correlation_coefficient < 0.8 or mean_rank / mea

return {
    "metric_name": "Correlation Coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}
```

```python
if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_hol
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2886fc8f.py", line 107, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2886fc8f.py", line 72, in run_trial
    circuit = generate_boolean_circuit(n)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2886fc8f.py", line 25, in generate_bool
    left = generate_boolean_circuit(n // 2)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2886fc8f.py", line 27, in generate_bool
    return [(x and y) for x in left] + [(x or y) for y in right]
                    ^
NameError: name 'y' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which means that it was unable to complete the required computations to verify the conjecture. | next: Re-run the test with proper error handling and ensure that it completes without crashing. If the test passes, re-evaluate the conjecture based on the results.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14209
2	propose	ollama_remote	glm4:latest	0	0	13985
3	preregistration	ollama_remote	glm4:latest	0	0	10167
4	novelty	ollama_remote	glm4:latest	0	0	8397
5	novelty	ollama_remote	glm4:latest	0	0	8763
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15122
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9282
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10947
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14601
10	judge	ollama_remote	glm4:latest	0	0	17678

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 123151 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/f3f8847f2523.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f3f8847f2523.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/f3f8847f2523.tar.gz` (if generated)